



Obsidian Brain

Guida alla Configurazione e all'Esecuzione

Istruzioni passo-passo per installare, configurare ed eseguire il
progetto da zero

1. Prerequisiti

Prima di iniziare, assicurati di avere installato sul tuo computer:

- **Python 3.10 o superiore** (verificabile con il comando `python --version` da terminale).
- **Ollama**, il programma che fa girare il modello linguistico in locale — scaricabile gratuitamente da <https://ollama.com>.
- Una **connessione internet attiva**, necessaria solo la prima volta, per scaricare le librerie Python e il modello di linguaggio (circa 2 GB in totale).
- Circa **3 GB di spazio libero** su disco (modello di embedding, modello Ollama, librerie Python).
- Facoltativo: PyCharm o un altro editor/IDE Python, se si vuole esplorare o modificare il codice.

2. Ottenere il progetto

Scarica o clona il progetto dalla repository GitHub in una cartella a tua scelta. Se hai Git installato, da terminale:

```
git clone <URL-della-repository>
cd obsidian-brain
```

In alternativa, scarica lo ZIP del progetto da GitHub (pulsante "Code" → "Download ZIP") ed estrailo in una cartella.

3. Creare l'ambiente virtuale

Si consiglia di lavorare all'interno di un ambiente virtuale Python (virtual environment), per tenere le librerie del progetto isolate da quelle installate globalmente sul sistema.

Da terminale, nella cartella del progetto:

```
python -m venv venv

# Attivazione su Windows:
venv\Scripts\activate

# Attivazione su macOS/Linux:
source venv/bin/activate
```

Se l'attivazione è andata a buon fine, il terminale mostrerà il prefisso `(venv)` prima del percorso.

4. Installare le librerie necessarie

Tutte le librerie richieste sono elencate nel file `requirements.txt`. Installale tutte insieme con:

```
pip install -r requirements.txt
```

Le principali librerie installate saranno:

- **sentence-transformers** — generazione degli embedding testuali
- **chromadb** — vector store locale e persistente
- **ollama** — comunicazione con il modello linguistico locale
- **fastapi** e **uvicorn** — layer API e server web
- **pytest** — esecuzione della suite di test automatici

Nota su Python 3.14: alcune librerie potrebbero impiegare qualche minuto in più del solito durante l'installazione, poiché per una versione di Python molto recente pip deve talvolta cercare tra più versioni prima di trovare pacchetti già compilati compatibili. È un comportamento atteso, non un errore.

5. Scaricare il modello linguistico locale

Il progetto usa Ollama per generare le risposte, interamente in locale. Scarica il modello (operazione da fare una sola volta, richiede internet):

```
ollama pull llama3.2
```

Verifica che il modello risponda correttamente:

```
ollama run llama3.2 "Ciao, funzioni?"
```

Se ottieni una risposta testuale, il modello è pronto. Dopo l'installazione, Ollama resta attivo in background e non richiede più connessione internet per funzionare.

Nota: il modello di embedding (`sentence-transformers`) viene invece scaricato automaticamente al primo avvio del progetto stesso, senza bisogno di comandi manuali aggiuntivi.

6. Eseguire i test automatici

Prima di usare il sistema, è consigliato verificare che tutto funzioni correttamente eseguendo la suite di test:

```
python -m pytest tests/ -v
```

Il progetto include 47 test automatici che coprono ogni componente della pipeline (parsing, chunking, embedding, retrieval, generazione risposta, API). Tutti i test usano dipendenze finte al posto di modelli o servizi reali, quindi vengono eseguiti in pochi secondi senza bisogno di internet o di Ollama attivo.

7. Avviare l'applicazione

Con Ollama attivo in background, avvia il server API:

```
python -m obsidian_brain.api
```

Il terminale mostrerà un messaggio simile a `Uvicorn running on http://127.0.0.1:8000`. Al primo avvio, il sistema indicizza automaticamente la vault di esempio inclusa nel progetto (`demo_vault/`) — l'operazione richiede qualche secondo.

Apri quindi un browser e visita l'indirizzo:

```
http://127.0.0.1:8000/docs
```

Comparirà la documentazione interattiva dell'API (Swagger UI), da cui puoi testare direttamente l'endpoint `/ask` senza bisogno di altri strumenti.

8. Usare la propria vault Obsidian

Per indicizzare una vault diversa da quella di esempio, imposta la variabile d'ambiente `VAULT_PATH` prima di avviare il server:

```
# PowerShell
$env:VAULT_PATH = "C:\percorso\della\tua\vault"
python -m obsidian_brain.api

# cmd
set VAULT_PATH=C:\percorso\della\tua\vault
python -m obsidian_brain.api

# macOS/Linux
export VAULT_PATH="/percorso/della/tua/vault"
python -m obsidian_brain.api
```

Il sistema funziona con qualsiasi cartella contenente file `.md` in formato Obsidian (frontmatter YAML, wikilink `[[nota]]`, tag `#tag` opzionali).

9. Arrestare l'applicazione

Per fermare il server, torna al terminale in cui è in esecuzione e premi `Ctrl+C`. Per uscire dall'ambiente virtuale attivato, digita `deactivate`.

10. Risoluzione dei problemi più comuni

Problema	Possibile causa e soluzione
<code>pytest</code> non riconosciuto come comando	Lo script non è nel PATH di sistema, comportamento comune su Windows. Usa <code>python -m pytest tests/ -v</code> invece di <code>pytest tests/ -v</code> .
Errore di compilazione durante <code>pip install</code> (es. libreria <code>tokenizers</code>)	Può accadere con versioni molto recenti di Python se pip prova a compilare una libreria da sorgente invece di scaricare un pacchetto già pronto. Verifica che <code>requirements.txt</code> non blocchi una versione troppo vecchia della libreria coinvolta (usa <code>>=</code> invece di <code>==</code> se necessario).
Le risposte sono sempre "Non ho trovato informazioni pertinenti"	Verifica che la cartella indicata in <code>VAULT_PATH</code> contenga effettivamente file <code>.md</code> , e che Ollama sia in esecuzione. Puoi usare <code>python -m obsidian_brain.inspect_index --vault TUA_VAULT</code> per verificare quante note e chunk sono stati indicizzati.
La pagina <code>/docs</code> non si apre	Il server non è in esecuzione, oppure è stato chiuso. Verifica che il terminale mostri il messaggio "Uvicorn running" e che tu stia usando la porta corretta (8000 di default).
Ollama non risponde / errore di connessione	Assicurati che l'applicazione Ollama sia installata e in esecuzione in background (di solito parte automaticamente dopo l'installazione). Prova <code>ollama run llama3.2 "test"</code> da terminale per verificare.
Un file Markdown della vault non viene letto correttamente	Verifica che il file sia salvato con codifica UTF-8 e a-capo standard. File salvati con strumenti che appiattiscono le righe in un'unica linea possono impedire il riconoscimento del blocco frontmatter.

Per approfondimenti su scelte tecniche, architettura e limiti noti del sistema, consultare il documento "**Documentazione Tecnica**" allegato al progetto.